Introduction to Deep Learning

Session 7: Generalization & Regularization

May 2026

Magda Gregorová - magda.gregorova@thws.de

Lecture Overview

thws
Technical University
of Applied Sciences
Würzburg-Schweinfurt

Weight Decay

thws
Technical University of Applied Sciences Würzburg-Schweinfurt

Explicit regularisation: add penalty term to the loss

$$\tilde{\mathcal{L}}(\boldsymbol{\theta}) = \mathcal{L}(\boldsymbol{\theta}) + \frac{\lambda}{2}\|\boldsymbol{\theta}\|^2$$

Large weights $\Rightarrow$ model relies heavily on specific features $\Rightarrow$ high variance

L2 regularisation (weight decay)

- Penalises $\|\boldsymbol{\theta}\|^2$ - shrinks all weights
- Smooth, differentiable
- `weight_decay` in `torch.optim`

L1 regularisation

- Penalises $\|\boldsymbol{\theta}\|_1$ - promotes sparsity
- Some weights go exactly to zero
- Useful for feature selection

Hyperparameter: $\lambda > 0$ controls strength - tuned on validation set

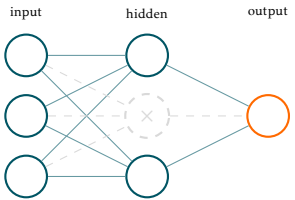
Deep NNs: baseline, combined with other techniques - less central than in classical ML

Dropout - Training

thws
Technical University
of Applied Sciences
Würzburg-Schweinfurt

During training: randomly zero each activation with probability p

- Each forward pass uses a different subnetwork
- Neurons cannot rely on specific others always being present
- Prevents co-adaptation of neurons



Typical values: $p = 0.1–0.5$ - tuned on validation set

Dropout - Evaluation

thws Technical University of Applied Sciences Würzburg-Schweinfurt

During evaluation: all neurons active - outputs scaled by $(1 - p)$ to compensate

`model.train()` vs `model.eval()`:

- `nn.Dropout` - zeroing active in train mode, disabled in eval mode
- Most layers (`nn.Linear`, `nn.ReLU`, ...) - unaffected
- Some other layers also have dual behaviour - covered later

Correct pattern:

```
model.train()
for X_batch, y_batch in train_loader:
    ... # training step

model.eval()
with torch.no_grad():
    ... # validation
```

Forgetting `model.eval()` is a silent bug - no error, wrong predictions

Dropout - Ensemble Interpretation

thws
Technical University of Applied Sciences Würzburg-Schweinfurt

Why does dropout work?

With n neurons, dropout implicitly trains 2^n different subnetworks - one per possible dropout mask

- Each mini-batch trains different subnetwork
- Subnetworks share weights - efficient training
- At test time: full network approximates **averaging** over all subnetworks

Ensembles generalise better than single models - averaging reduces variance

Dropout achieves ensemble-like generalisation at the cost of a single model

Data Augmentation

thws
Technical University of Applied Sciences Würzburg-Schweinfurt

Idea: apply label-preserving transformations to training examples to create new ones

Key principle - invariance: the transformation must not change the label

- Rotate image of cat $\Rightarrow$ still cat
- Add small noise to patient measurements $\Rightarrow$ same diagnosis
- Model should be invariant to these transformations - augmentation enforces it

Examples:

- **Images:** random flips, rotations, crops, colour jitter
- **Tabular data:** Gaussian noise on features, random feature masking

Domain knowledge required - only you know which transformations preserve the label